Google Summer of Code 2025

Context-Aware Autocomplete and Refactoring Enhancements for the p5.js Web Editor

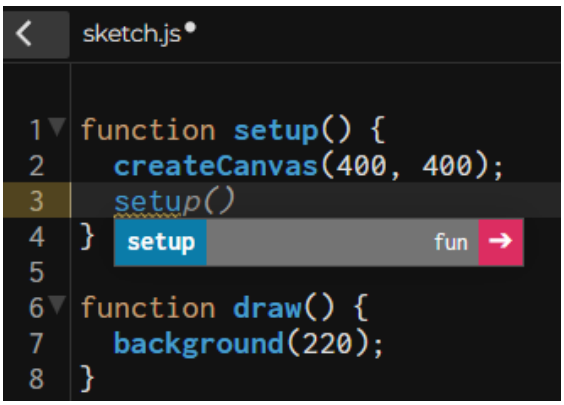
Proposal Title: Context-Aware Autocomplete and Refactoring Enhancements for the p5.js Web Editor

Project: User-Friendly Features in the p5.js Editor

Name: Kamakshi

Overview

Currently, the autocomplete feature in the p5.js Web Editor is facilitated by CodeMirror and a custom-built static hinter (as given in the structured JSON file, p5-hinter.js). The Hinter suggests all the available functions, and while it is helpful, the current implementation lacks context-awareness, and that can be confusing, especially for beginners.



For example, a function like setup() is meant to be defined only once in a sketch, but it still shows up as a suggestion even when you're typing inside another setup() or draw() block. This makes the experience feel less intuitive and can mislead someone who's just starting out.

A beginner might think they're supposed to write setup() again, or not understand when and where specific functions should be used.

A few other examples of this are:

- Calling createCanvas() inside draw(): The Hinter might suggest createCanvas() even inside the draw() function. A beginner might think it's okay to call it repeatedly, not realizing it's meant to be in setup() to initialize the canvas once.
- Suggesting mousePressed() inside another function body: mousePressed() is a callback function, not one that should be nested. So suggesting it inside setup() or any user-defined function can lead to confusion about its usage.
- Suggesting functions like preload(), setup(), or draw() inside class definitions or object literals might lead to non-functional code and misunderstanding of how p5's global mode works.

That is why context-aware suggestions are so important. The editor should be smart enough to understand where the user is typing and what functions or variables make sense in that context. This creates a smoother experience, reduces confusion, and makes the Web Editor feel more intelligent and helpful, which aligns with Processing Foundation's mission to make creative coding more accessible and beginner-friendly.

This problem has been highlighted in the 2017 GSoC Proposal as well (I will be elaborating on that later in the technical implementation section).

Further, this issue #3078 raised will also be tackled, which outlines the following feature enhancements:

- Add autocomplete hints for variables, functions, and classes
- Add an option to access the documentation for interacting with code belonging to p5.js

- Add an option to jump to the variable, function, or class when interacting with code belonging to the sketch itself

To build on that, I also want to implement context-aware renaming. I believe it is an important feature that any code editor should have as it saves a ton of time and helps prevent bugs caused by missed renames or accidental overwrites. While we do have the find-and-replace feature in p5.js Web Editor, context-aware renaming is obviously a smarter and more efficient way to update code.

Project Goals

With that being said, here is what I plan to accomplish with this project:

- Implement context-aware suggestions in the autocomplete hinter
- Add all the features outlined in Issue #3078
- Implement context-aware renaming for variables, functions, and classes

Technical Implementation Details

Context aware suggestions in the autocomplete hinter

Back in 2017, the GSoC proposal by Kagan explored using Tern for implementing autocomplete. One of my initial ideas was to revisit that direction by customizing the Tern plugin to improve its understanding of scope in p5.js code as it had a scoping problem initially. However, the issue with Tern is that it's outdated, inefficient for modern use cases, and difficult to extend. Plus, it hasn't been actively maintained over the years.

I also came across the idea of running a JavaScript/TypeScript LSP as a backend for more intelligent completions (source: discourse thread). While it's an interesting and powerful approach, I decided not to go with it for this proposal since the p5.js Web Editor is currently fully browser-based. Introducing a backend just for autocomplete might change this architecture and increase maintenance complexity. That said, if mentors feel this is a direction worth exploring, I'd be happy to look into it.

Now, given that user experience, usability, and long-term maintainability are key priorities, a better approach would be to use Acorn or @babel/parser along with eslint-scope. This will ensure that the autocomplete hinter remains fast, efficient, and easier to maintain or expand as the project grows.

Acorn is a tool that takes JavaScript code and breaks it down into a structured format called an Abstract Syntax Tree (AST). I have also considered using @babel/parser, which is more powerful but I will have to make sure to adapt the AST it generates (Babel AST format) to match with what eslint-scope expects, which is an ESTree. Acorn produces an AST that is compatible with ESTree.

Now this is how the AST of a simple program written in p5.js looks like using acorn & @babel/parser:

```
function setup() {
  createCanvas(400, 400);
}

function draw() {
  background(220);
}
```

```
- Program {
    type: "Program"
    start: 0
    end: 86
  - body: [
    + FunctionDeclaration {type, start, end, id, expression, ... +4}
    + FunctionDeclaration {type, start, end, id, expression, ... +4}
    ]
    sourceType: "module"
  }
```

(acorn parser)

```
- File {
    type: "File"
    start: 0
    end: 88
  + loc: {start, end, filename, identifierName}
    errors: [ ]
  - program: Program {
      type: "Program"
      start: 0
      end: 88
    + loc: {start, end, filename, identifierName}
      sourceType: "module"
      interpreter: null
    - body: [
      + FunctionDeclaration {type, start, end, loc, id, ... +4}
      + FunctionDeclaration {type, start, end, loc, id, ... +4}
      ]
      directives: [ ]
    }
    comments: [ ]
  }
```

(@babel/parser)

The AST of the following code would be:

```
function draw() {
  function setup() {
    createCanvas(400, 400);
```

```
  }
  background(220);
}
```

```
- Program {
    type: "Program"
    start: 0
    end: 92
  - body: [
    - FunctionDeclaration {
        type: "FunctionDeclaration"
        start: 0
        end: 91
      + id: Identifier {type, start, end, name}
        expression: false
        generator: false
        async: false
        params: [ ]
      - body: BlockStatement {
          type: "BlockStatement"
          start: 16
          end: 91
        - body: [
          - FunctionDeclaration {
              type: "FunctionDeclaration"
              start: 20
              end: 70
```

```
- File {
    type: "File"
    start: 0
    end: 92
  + loc: {start, end, filename, identifierName}
    errors: [ ]
  - program: Program {
      type: "Program"
      start: 0
      end: 92
    + loc: {start, end, filename, identifierName}
      sourceType: "module"
      interpreter: null
    - body: [
      - FunctionDeclaration = $node {
          type: "FunctionDeclaration"
          start: 0
          end: 91
        + loc: {start, end, filename, identifierName}
        + id: Identifier {type, start, end, loc, name}
          generator: false
          async: false
          params: [ ]
        - body: BlockStatement {
            type: "BlockStatement"
            start: 16
            end: 91
          + loc: {start, end, filename, identifierName}
          - body: [
              + FunctionDeclaration {type, start, end, loc, id, ... +4}
```

(The ASTs have been generated from https://astexplorer.net. Left image: acorn, right image: @babel/parser)

So, now we can see: a FunctionDeclaration named "setup" exists inside the body of another FunctionDeclaration named "draw". In order to detect this programmatically, we can simply parse the code and traverse the tree using acorn-walk, so a very simple code for this would be:

```
import { parse } from "acorn";
import * as walk from "acorn-walk";

const ast = parse(userCode, { ecmaVersion: 2020});

walk.ancestor(ast, {
  FunctionDeclaration(node, ancestors) {
    if (node.id.name === "setup") {
      const parent = ancestors[ancestors.length - 2];
      if (parent.type === "FunctionDeclaration" && parent.id.name === "draw") {
        console.warn("setup() is declared inside draw()");
      }
    }
  }
});
```

Similarly, we can do the same using @babel/parser instead of Acorn. It also gives us an AST that we can walk through to check where each function is defined:

```
const parser = require("@babel/parser");
const traverse = require("@babel/traverse").default;

const ast = parser.parse(userCode, {
  sourceType: "module",
});

traverse(ast, {
  FunctionDeclaration(path) {
    if (path.node.id.name === "setup") {
      const parent = path.findParent(p => p.isFunctionDeclaration());
      if (parent && parent.node.id.name === "draw") {
        console.log("setup() is declared inside draw()");
      }
    }
  }
});
```

And in order to prevent suggesting setup inside draw altogether, we can simply get the current cursor position, scan the AST to find out which block/function the user is currently inside, and if that block is the draw() function, we can apply a blacklist as follows:

```
{
    "draw": {
      "blacklist": ["setup", "preload"]
    }
}
```

During suggestion rendering, we check the parent context and the current function name, and subsequently remove any blacklisted ones from the suggestion pool.

Blacklist & Whitelist

This approach requires the use of maintaining blacklists and whitelists for some functions, especially preload(), setup(), and draw().

- The whitelist defines which functions are allowed in a given context
- The blacklist defines functions that should not be used in a certain context

With Acorn or @babel/parser, we parse the code into an AST and determine which function/block (like setup, draw, or user-defined functions) the cursor is in. From that block

name, we look up a whitelist and show only those functions or a blacklist and exclude forbidden ones. So programmatically, it would look like:

```
if (whitelist[context]) {
  suggestions = suggestions.filter(s => whitelist[context].includes(s));
}
if (blacklist[context]) {
  suggestions = suggestions.filter(s => !blacklist[context].includes(s));
}
```

To address the main point here now– creating and maintaining these lists efficiently:
Instead of doing it manually for every p5.js function, we can parse the p5.js reference and generate scope tags, for which I will write a script. My plan is to go with a hybrid approach, which entails combining:

1. Auto-generated lists from parsing p5.js documentation
2. Manually written ones for special edge cases or overrides

So this way, we will be able to automate 90% of the work whilst also having full control over exceptions and any other cases.

To maintain and update these lists, I plan to keep them as separate, human-readable JSON files which can be updated alongside any new p5.js releases. I will also add clear documentation for this.

Also just as a note here, the reason eslint-scope isn't being used right here is that while eslint-scope excels at static analysis (renaming, jump-to-definition), it doesn't expose enough parent/ancestor structure to help with suggestions where syntactic context matters. It only provides scope data.

Add autocomplete hints for variables, functions, and classes

Once Acorn or @babel/parser gives the AST, eslint-scope takes it and figures out which variables/functions are in scope at a certain place in the code and where each variable/function is defined and used.

Hence, after parsing the AST plus running eslint-scope, each Scope will have:

- .variables → declared identifiers (with name, kind, defs)
- .references → where they're used

So, we can just use the current cursor position to find which scope we are in, and offer suggestions from that scope's variables.

In order to ensure that this approach doesn't lead to lags and maintains the editor's performance, I plan to optimize it by experimenting with a few techniques:

- Debouncing: re-parse the AST only after a short pause in typing, instead of doing it on every keystroke. There is also a discussion in CodeMirror's forum that shows how debouncing improved performance, which validates it as an effective optimization technique for p5.js Web Editor.
- Scope caching: we can cache known scopes (functions, variables, etc.) and only rebuild them if the cursor moves into a different block
- Another advanced strategy that can be used here is incremental parsing, where instead of re-parsing everything, we detect the code changes and update only the affected subtree. This will require a deeper understanding of AST structures.

Add an option to jump to the variable, function, or class

The goal here is that when a user Ctrl+clicks or right-clicks "Go to Definition" on a variable/function/class, the editor should scroll to (and maybe highlight) where that was defined. Potentially, this feature can also be accessed via a right-click context menu.

The technical implementation for this will be:

- We get the current identifier under the cursor using CodeMirror's cursor position API to extract the word or token the user clicked
- After parsing the AST and running eslint-scope, each scope will then have a list of references
    - Every reference has a .resolved property (if it's defined).
    - .resolved.defs[0].node gives the definition node
- And so from that, we find the corresponding definition node
- Use CodeMirror's setCursor or scrollIntoView to jump there. And we can also use markText() to highlight that line temporarily.

Enable context-aware renaming

The goal here is that when a user wants to rename a variable, function, or class, only the relevant references (in the same scope or valid referencing context) should be renamed.

For example, if two functions each have a variable named speed, renaming one should not rename the other. Again, this feature can be listed in the right-click context menu.

The technical implementation is as follows:

- Parse the code using acorn or @babel/parser
- Run eslint-scope on the parsed AST to get a ScopeManager which contains:
    - all the scopes
    - each scope has .variables and .references
- Then, we find the name and exact position of the variable under the cursor by walking through the ScopeManager to find the scope that contains the cursor

- And finally, we find the right reference (matching scope and name) by:
    - using scope.references to match the variable under the cursor
    - using .resolved on that reference to get the variable object
- Update all references belonging to that variable

Final Approach

There are two approaches I'm planning to experiment with:

1. Use Acorn and eslint-scope
2. Use @babel/parser and eslint-scope

The p5.js Web Editor already uses Babel so reusing its parser can be better in the sense that it maintains consistency. However, the problem here is that Babel's parser is more complex because it handles a wide range of transformations and syntax extensions. And this can introduce performance overhead.

Acorn is minimalistic and fast, especially for parsing large files or performing real-time analysis, and because we want the editor to have no lags, it is important that we experiment with both and see which one fits best.
(some sources include: https://github.com/babel/babel/issues/3921, https://npm-compare.com)

I've also considered the upcoming release of p5.js 2.0 and will design the autocompletion features to remain compatible with both 1.x and 2.0. Additionally, I'll stay updated with changes in the 2.0 RFC process and plan my implementation in a way that it can easily adapt as p5.js evolves.

Ensuring Digital Accessibility

As the p5.js Web Editor serves a wide range of learners, including those new to programming, accessibility is a core concern. Throughout this project, I will ensure that all features are compatible with screen readers, navigable with a keyboard, and adhere to WCAG 2.1 AA guidelines. I will also ensure reduced cognitive load by offering clear, minimal suggestion UIs.

So my technical implementation will also extend to the following:

1. Context-Aware Suggestions
    - Ensure suggestions are navigable via keyboard (Tab/Arrow keys)
    - Announce suggestions with a live region (aria-live="polite") so screen readers can read them

2. User-defined Identifiers and Scope-Aware Filtering
    - Avoid clutter by limiting suggestions intelligently to reduce cognitive load

- Group suggestions with headings (aria-label="Local Variables", aria-label="p5.js Functions")

3. Jump-to-Definition
   - Show a toast or label like “Jumped to: line 12” for screen reader users
   - Ensure the target definition receives focus visually and programmatically

4. Context-aware renaming
   - Add a confirmation dialog with screen reader labels
   - Use focus traps inside dialogs to help keyboard-only users
   - Ensure all dialog actions are accessible via keyboard shortcuts

Evaluation Metrics

These are the potential metrics for the autocomplete and navigation features that I will be considering:

Metric	**Potential testing method**
AST parse time after each user edit	performance.now() around the parsing functions
Suggestion latency as in the time between keypress & dropdown render	requestAnimationFrame to measure UI response
Memory used during large sketches	Chrome DevTools > performance tab & profiler
Delay while typing continuously	Use Chrome’s lighthouse & DevTools timeline panel

For accessibility testing, the metrics I have defined are:

Metric	**Potential testing method**
Screen reader behavior of dropdowns, tooltips, etc.	Test with NVDA, VoiceOver, or ChromeVox on different platforms
Full keyboard accessibility	Manually test keyboard-only navigation for all elements
Use of correct ARIA roles and properties on UI components	Use axe-core or Chrome Lighthouse accessibility audits
UI readability in high contrast or forced color modes	Enable high contrast mode and test visibility manually
Whether animations respect reduced	Enable reduced motion in system settings and

motion system setting	verify that transitions are minimal

I'm open to refining these metrics as I learn more and receive guidance from my mentors.

If time permits

If time permits, I would also love to implement the following features. All of them described below can be in the form a tooltip that can be enabled or disabled via settings, depending on user preference.

1. **Right-Side Inline Descriptions for Suggestions (Configurable)**

Each suggestion in the dropdown will include a short description on the right side, summarizing what the function does. These descriptions will be fetched from the p5.js reference documentation.

As of now, there is a pink arrow that redirects the user to a new browser tab, but I believe this new feature would save users time and avoid context switching.

2. **On-Hover Detailed Descriptions (Configurable)**

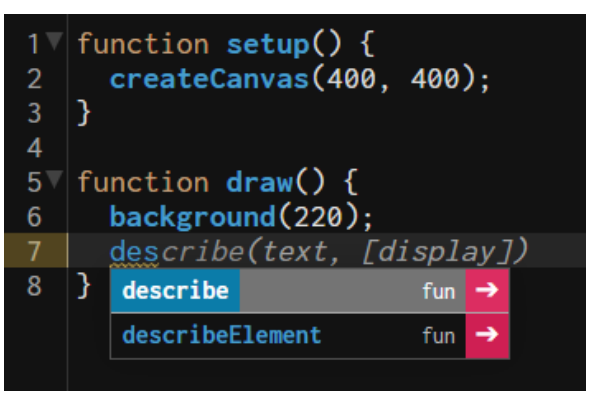
Users will be able to hover over a p5.js function to view a detailed description of it, including usage examples and links to the official p5.js reference page.

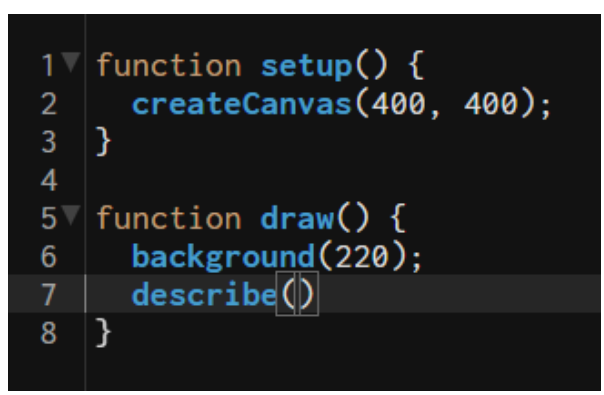
Example (on hover):
createCanvas(width, height): Sets the size of the canvas. This function should only be called once inside setup().

This will help balance a clean interface with access to deeper documentation, improving both beginner and advanced user experiences.

3. **Parameter Highlighting in Suggestions (Configurable)**

Each suggestion will display the function signature with its parameters highlighted in a soft, light grey color. As of now, it is shown only while typing but disappears as soon as the user selects it:





I believe it would be helpful if the parameters are still shown even after the user has selected the function.

Timeline

I plan to complete all the tasks before the GSoC 2025 coding period ends. Apart from this, I'll also write medium blogs, reporting my progress and the experiences I had while developing the features.

Community Bonding Period (May 8 – June 1)
During this phase, my focus will be on preparation and deep understanding of the project's scope. I plan to:

- Actively engage with mentors, ask questions, and get feedback on my approaches
- Build more sketches with p5.js to get more familiar with how the library works in practice, especially with regard to context-aware usage of functions
- Create a consolidated list of functions that should and shouldn't be suggested in specific contexts (blacklist/whitelist), and get this reviewed by mentors

Coding Period Timeline:

Week	**Objective**
1 (June 2 – June 8)	➔ Set up parsing pipeline using acorn & @babel/parser in the p5.js Web Editor ➔ Parse user code & generate AST on every code change for both acorn & @babel/parser
2 (June 9 – June 15)	➔ Build a prototype for context-aware suggestions using AST and manually defined whitelist/blacklist JSON for both acorn & @babel/parser ➔ Filter suggestions based on cursor position and scope ➔ Implement basic debouncing logic to avoid reparsing on every keystroke
3 (June 16 – June 22)	➔ Begin identifying user-defined code using both acorn & @babel/parser along with eslint-scope & suggest them in autocomplete ➔ Build on that & implement jump-to-definition using both ➔ Update mentors and make sure to take their suggestions
4 (June 23 – June 29)	➔ Work on context-aware renaming feature ➔ After experimenting with both acorn and @babel/parser, assess best possible approach using evaluation metrics ➔ Continuously take feedback from mentors at this point &

	make modifications accordingly
5 (June 30 – July 6)	➔ Expand whitelist/blacklist to include more p5.js-specific context rules by parsing the p5.js reference ➔ Create a utility to allow easy editing or adding of new rules in the future ➔ Add logic for edge cases where context isn't determinable ➔ Discuss with mentors about edge cases or more advanced contextual behaviors + work on their suggestions
6 (July 7 – July 13)	➔ Experiment with scope caching for optimization ➔ Learn more about incremental parsing and implement the same ➔ Test on small and large sketches to catch performance issues early ➔ Decide best possible optimization mechanism ➔ Take suggestions from mentor and peers
7 (July 14 – July 20)	*Mentor feedback and mid-term evaluation* ➔ Work to maintain smooth UI ➔ Perform final checks ➔ Submit mid-term evaluation
8 (July 21 – July 27)	➔ Work on mentor feedback ➔ Buffer time for other modifications ➔ Learning basics of ARIA, WCAG & accessibility practices ➔ Write blog about my progress so far
9 (July 28 – Aug 3)	➔ Implement ARIA and keyboard support across features ➔ Reduced motion preferences ➔ Time for other modifications as well
10 (Aug 4 – Aug 10)	➔ Perform accessibility testing & check WCAG 2.1 AA compliance ➔ Debugging with screen readers, keyboard navigation, contrast, etc. ➔ Get feedback from mentor & work on modifications
11 (Aug 11 – Aug 17)	➔ Iterative feedback + fix issues found during testing ➔ Buffer time for any other modifications ➔ Start working on additional features if time permits ➔ Write blog about my progress so far
12 (Aug 18 – Aug 24)	➔ Work on final submission and final project report ➔ Finish any documentation

Deliverables

By the end of the mid-term evaluation, I aim to deliver:

- An improved autocomplete system that provides context-aware suggestions and user-defined variables, functions, and classes.
- A feature that allows jumping to the definition of user-defined code elements.
- Support for context-aware renaming of variables, functions, and classes within a sketch.
- A pull request integrating these changes into the codebase

By the final evaluation, I aim to deliver:

- Incorporation of all feedback received during the mid-term evaluation.
- Accessibility improvements including ARIA attributes, keyboard navigation, and support for reduced motion preferences.
- Thorough testing for all features, along with clear documentation of the tests.
- A clean UI/UX for all newly added features.
- A final pull request integrating all changes and features.

Related Work

These are some related PRs I came across:

1. Autocomplete Hinter original: implements a basic autocomplete using CodeMirror's hinter addon with p5.js keywords and fuzzy matching via Fuse.js. My proposal builds on this by adding context-aware suggestions & user-defined completions.
2. Autocomplete Hinter ++: introduced a Copilot-style inline completion preview. I plan to enhance it further as mentioned in point 3 of the if time permits section.
3. Enhanced AutoComplete Hinter: I've taken the feedback from this PR into account. The hinter will only trigger in contextually appropriate situations based on AST analysis, and I'll ensure the UX is non-disruptive. I have also addressed the issues that this PR was meant for.

Availability

My main plan for the summer has been GSoC itself. I'm really excited about this project and the opportunity to contribute to the p5.js Web Editor in a meaningful way. Since this is my primary focus, I'll be fully available to dedicate time to the project throughout the entire coding period.

I've always been diligent with finishing work on time and managing my schedule well. I'll make sure to stick to the proposed timeline and maintain clear communication with my mentors. I'm confident I can complete the project as planned, and I'm looking forward to the learning experience that comes with it.

Other GSoC proposals

This is the only proposal that I am submitting for Google Summer of Code 2025.